

Research Notes on Momentum optimization

Momentum optimization

>> Section 1

The scaling factor $\xi^* \in \mathbb{R}$ can be found through the bisection method since in most regular models, such as the aforementioned generalized linear models, function $q(\cdot)$ is decreasing, and thus the search bounds for ξ^* are $[\min(0, f(0)), \max(0, f(0))]$.

>> Section 2

where ε is a small scalar (e.g. 10^{-8}) used to prevent division by 0, and β_1 (e.g. 0.9) and β_2 (e.g. 0.999) are the forgetting factors for gradients and second moments of gradients, respectively. Squaring and square-rooting is done element-wise. As the exponential moving averages of the gradient $m_w(t)$ and the squared gradient $v_w(t)$ are initialized with a vector of 0's, there would be a bias towards zero in the first training iterations. A factor $\frac{1}{1 - \beta_1}$ is introduced to compensate this bias and get better estimates $\hat{m}_w(t)$ and $\hat{v}_w(t)$. The initial proof establishing the convergence of Adam was incomplete, and subsequent analysis has revealed that Adam does not converge for all convex objectives. Despite this, Adam continues to be used due to its strong performance in practice.

>> Section 3

Averaging Averaged stochastic gradient descent, invented independently by Ruppert and Polyak in the late 1980s, is ordinary stochastic gradient descent that records an average of its parameter vector over time. That is, the update is the same as for ordinary stochastic gradient descent, but the algorithm also keeps track of

>> Section 4

$w := w - \eta \nabla Q(w) = w - \eta \sum_{i=1}^n \nabla Q_i(w)$.

>> Section 5

Further proposals include the momentum method or the heavy ball method, which in ML context appeared in Rumelhart, Hinton and Williams' paper on backpropagation learning and borrowed the idea from Soviet mathematician Boris Polyak's 1964 article on solving functional equations. Stochastic gradient descent with momentum remembers the update Δw at each iteration, and determines the next

update as a linear combination of the gradient and the previous update:

>> Section 6

Sign-based stochastic gradient descent Even though sign-based optimization goes back to the aforementioned Rprop, in 2018 researchers tried to simplify Adam by removing the magnitude of the stochastic gradient from being taken into account and only considering its sign. This results in a significantly lower communication cost of transferring gradients from workers to the parameter server. In this sense, it serves to better compress the gradient information, while having comparable convergence to standard SGD.

>> Section 7

AdaGrad AdaGrad (for adaptive gradient algorithm) is a modified stochastic gradient descent algorithm with per-parameter learning rate, first published in 2011. Informally, this increases the learning rate for sparser parameters and decreases the learning rate for ones that are less sparse. This strategy often improves convergence performance over standard stochastic gradient descent in settings where data is sparse and sparse parameters are more informative. Examples of such applications include natural language processing and image recognition. It still has a base learning rate η , but this is multiplied with the elements of a vector $\{G_{j,i}\}$ which is the diagonal of the outer product matrix

>> Section 8

$w_{\text{new}} := \arg \min_w \{ Q(w) + \frac{1}{2} \eta \|w - w_{\text{old}}\|^2 \}$.

>> Section 9

$m_w(t) := \beta_1 m_w(t-1) + (1 - \beta_1) \nabla w_L(t-1)$

>> Section 10

A compromise between computing the true gradient and the gradient at a single sample is to compute the gradient against more than one training sample (called a "mini-batch") at each step. This can perform significantly better than "true" stochastic gradient descent described, because the code can make use of vectorization libraries rather than computing each step separately as was first shown in where it was called "the bunch-mode back-propagation algorithm". It may also result in smoother convergence, as the gradient computed at each step is

Research Notes on Momentum optimization

Momentum optimization

averaged over more training samples. The convergence of stochastic gradient descent has been analyzed using the theories of convex minimization and of stochastic approximation. Briefly, when the learning rates η decrease with an appropriate rate, and subject to relatively mild assumptions, stochastic gradient descent converges almost surely to a global minimum when the objective function is convex or pseudoconvex, and otherwise converges almost surely to a local minimum. This is in fact a consequence of the Robbins–Siegmund theorem.

>> Section 11

Stochastic gradient descent (often abbreviated SGD) is an iterative method for optimizing an objective function with suitable smoothness properties (e.g. differentiable or subdifferentiable). It can be regarded as a stochastic approximation of gradient descent optimization, since it replaces the actual gradient (calculated from the entire data set) by an estimate thereof (calculated from a randomly selected subset of the data). Especially in high-dimensional optimization problems this reduces the very high computational burden, achieving faster iterations in exchange for a lower convergence rate. The basic idea behind stochastic approximation can be traced back to the Robbins–Monro algorithm of the 1950s. Today, stochastic gradient descent has become an important optimization method in machine learning.

>> Section 12

where E denotes taking the expectation with respect to the random choice of indices in the stochastic gradient descent scheme. Since this approximation does not capture the random fluctuations around the mean behavior of stochastic gradient descent solutions to stochastic differential equations (SDEs) have been proposed as limiting objects. More precisely, the solution to the SDE

>> Section 13

RMSProp has shown good adaptation of learning rate in different applications. RMSProp can be seen as a generalization of Rprop and is capable to work with mini-batches as well opposed to only full-batches.